



Facultad de Ingeniería

Magíster en Ciencia de Datos e Inteligencia Artificial

**SOBREDURACIÓN EN LA TITULACIÓN DE LA EDUCACIÓN SUPERIOR
CHILENA (2020-2025)**

Avance del proyecto: Fase 3 — Núcleo algorítmico, eficiencia e implementación
orientada a objetos

Informe de avance presentado para la asignatura Programación para la Ciencia de
Datos (202682.1927)

Autor:

Sebastian Antunez Noguera

Profesor:

Omar Salinas

Santiago, Chile

27 de septiembre de 2026

Ficha del trabajo

Antecedente	Detalle
Asignatura	Programación para la Ciencia de Datos (202682.1927)
Programa	Magíster en Ciencia de Datos e Inteligencia Artificial
Profesor	Omar Salinas
Autor	Sebastian Antunez Noguera
Entrega	Segundo avance sumativo — Fase 3
Repositorio	https://github.com/santunez75/proyecto-titulados (rama fase-3/nucleo-algoritmico)
Notebook de la fase	F3/F3_Nucleo_Algoritmico.ipynb
Insumo	Dataset analítico de la Fase 2: 1.246.760 titulados de pregrado, SIES 2020–2025
Entorno de ejecución	Python 3.14.7 · pandas 3.0.5 · NumPy 2.5.3 · Windows 11
Pruebas automatizadas	73 (22 del pipeline de F2 + 51 del núcleo de F3)

Todas las cifras, tablas y figuras de este informe fueron generadas por el notebook de la Fase 3 y son reproducibles ejecutándolo; sus salidas quedan en `reports/tables/f3_*.csv` y `reports/figures/f3_*.png`.

Índice

II. Introducción y contextualización	5
2.1 Contexto de la fase	5
2.2 Objetivo de la fase.....	5
2.3 Alcance	5
2.4 Conexión con las fases del proyecto	5
III. Mejoras integradas desde la Fase 2	7
IV. Arquitectura del núcleo algorítmico.....	8
4.1 Módulos y responsabilidades	8
4.2 Flujo de datos y dependencias	8
4.3 Cohesión, acoplamiento y escalabilidad.....	9
V. Codificación funcional: algoritmos estructurados y recursivos	10
5.1 Agregación en una pasada (programación estructurada)	10
5.2 Merge sort recursivo.....	10
5.3 Quickselect: la mediana sin ordenar	10
5.4 Búsqueda binaria recursiva y conteo por umbral	11
VI. Análisis de complejidad y eficiencia.....	12
6.1 Cotas teóricas	12
6.2 Método de medición	12
6.3 Comparación de implementaciones a tamaño fijo	12
6.4 Curvas de escalamiento y exponente empírico	13
6.5 Memoria y memorización	14
6.6 Interpretación y elección justificada	14
VII. Programación orientada a objetos y patrones de diseño	16
7.1 Encapsulamiento: Titulado	16
7.2 Protocolo de secuencia: Cohorte.....	16
7.3 Herencia y polimorfismo: Metrica (patrón Strategy)	17
7.4 Recursión sobre objetos: NodoJerarquia (patrón Composite).....	17
7.5 Herencia con propósito: Transformador y ModeloGradiente.....	18
7.6 Patrones de diseño y su función en la arquitectura	18
7.7 Diagrama de clases.....	19
VIII. Preprocesamiento y transformación para la modelación.....	20
8.1 Validaciones del resultado.....	20
IX. Modelación desde primeros principios.....	21
9.1 Descenso de gradiente.....	21

9.2 Convergencia verificada	21
9.3 Desempeño e interpretación.....	22
X. Validación técnica y verificación del código	24
10.1 Corrección contra referencias independientes.....	24
10.2 Pruebas automatizadas.....	24
10.3 Reglas de coherencia con la Fase 2.....	25
10.4 Ejecución de extremo a extremo	25
XI. Notebook F3_Nucleo_Algoritmico.ipynb	26
XII. Repositorio GitHub	27
12.1 Estructura de la Fase 3	27
12.2 Historial de commits de la fase.....	27
XIII. Conclusiones y proyección	29
13.1 Sobre el diseño algorítmico	29
13.2 Sobre la programación orientada a objetos	29
13.3 Sobre la modelación.....	29
13.4 Limitaciones y proyección	29
XIV. Bibliografía	31
Bibliografía docente.....	31
Documentación técnica oficial	31
Bibliografía complementaria	31
XV. Anexos	33
Anexo A. Instituciones con mayor sobreduración media en el árbol jerárquico.....	33
Anexo B. Salida completa de las pruebas automatizadas	33
Anexo C. Contribuciones y ramas	34
Anexo D. Reproducción de la fase	35

II. Introducción y contextualización

2.1 Contexto de la fase

La Fase 2 dejó un conjunto de datos analítico de 1.246.760 titulados de pregrado con la variable objetivo —la sobreduración, o semestres cursados por sobre la duración teórica del plan— reconstruida, validada y documentada. Sin embargo, todo el procesamiento descansaba en pandas: las agregaciones, los ordenamientos y los estadísticos de orden eran llamadas a una biblioteca cuyo funcionamiento interno se daba por supuesto, y las reglas del dominio vivían dispersas en funciones sobre DataFrames.

La Fase 3 construye el núcleo algorítmico del proyecto. Los algoritmos centrales se implementan desde primeros principios en Python, con programación estructurada y recursiva; se analiza su complejidad temporal y espacial y se mide su eficiencia con métodos formales frente a las implementaciones de pandas y NumPy; y el dominio se organiza con programación orientada a objetos, aplicando encapsulamiento, herencia, polimorfismo y patrones de diseño básicos. Como continuidad del plan trazado en la Fase 1, se implementan además dos modelos —regresión lineal y logística por descenso de gradiente— desde cero, contrastados con la solución exacta.

2.2 Objetivo de la fase

Diseñar e implementar el núcleo algorítmico del proyecto mediante programación estructurada, recursiva y orientada a objetos, evaluando formalmente su eficiencia computacional y documentando las decisiones de arquitectura, de modo que la solución resulte modular, reutilizable, verificable y escalable hacia la Fase 4.

2.3 Alcance

- **Incluye:** un paquete `src/nucleo` de nueve módulos, 51 pruebas automatizadas nuevas, un notebook ejecutado de corrido, mediciones reproducibles de tiempo y memoria, y dos modelos implementados desde primeros principios.
- **Excluye:** la selección definitiva de un modelo predictivo y la comunicación de resultados a público no técnico, que corresponden a la Fase 4. Los modelos de esta fase demuestran la implementación del algoritmo de optimización; su capacidad predictiva se interpreta pero no se optimiza.

2.4 Conexión con las fases del proyecto

Tabla 1. Articulación de la Fase 3 con las fases previas y posteriores

Fase	Aporte a la Fase 3	Lo que la Fase 3 entrega
F1	Entorno reproducible, paquete src/ modular y convenciones de código que esta fase extiende.	—
F2	Dataset analítico validado (insumo de todo el núcleo) y motor de reglas de validación, reutilizado aquí sin modificaciones.	Tres mejoras concretas sobre debilidades detectadas al reproducir la Fase 2 (sección III).
F4	—	Clases de dominio (Cohorte, jerarquía recursiva), transformadores y modelos base sobre los que se construirá el reporte analítico final.

III. Mejoras integradas desde la Fase 2

La reproducción del proyecto en un segundo computador, realizada al cierre de la Fase 2, dejó a la vista tres debilidades del código. Cada una se resolvió en esta fase; la tabla vincula el problema con la mejora y con el lugar del repositorio donde puede verificarse.

Tabla 2. Problemas detectados en la Fase 2 y mejoras aplicadas en la Fase 3

Problema detectado	Mejora aplicada	Dónde verificarla
La celda de pruebas usaba <code>sys.executable</code> : con un kernel de Jupyter distinto al entorno virtual fallaba sin mostrar la causa, porque el error iba por <code>stderr</code> .	La celda apunta explícitamente al intérprete del <code>.venv</code> del proyecto e imprime <code>stderr</code> cuando el código de salida no es cero.	F3, sección 8
Los notebooks dependían del Parquet completo (no versionado): en un equipo sin los CSV originales no podían ejecutarse.	<code>cargar_datos()</code> degrada a la muestra versionada de 5.000 registros con un aviso explícito, sin cambiar el resto del flujo.	<code>src/nucleo/fachada.py</code>
La lógica analítica estaba en funciones sueltas sobre DataFrames; los invariantes del dominio (una duración no puede ser cero ni negativa) no estaban garantizados en ninguna parte.	Modelo de dominio con Titulado y Cohorte: los invariantes se validan al construir el objeto y las magnitudes derivadas son propiedades de solo lectura.	<code>src/nucleo/dominio.py</code> ; F3, sección 5

Las tres mejoras comparten un criterio: cada una convierte un fallo silencioso en un comportamiento explícito. Un error que se ve es un error que se corrige; uno que no se ve contamina resultados.

IV. Arquitectura del núcleo algorítmico

4.1 Módulos y responsabilidades

El núcleo se organiza como un subpaquete `src/nucleo` con nueve módulos, cada uno con una única razón para cambiar. La tabla se corresponde con la que el notebook genera por introspección del propio paquete.

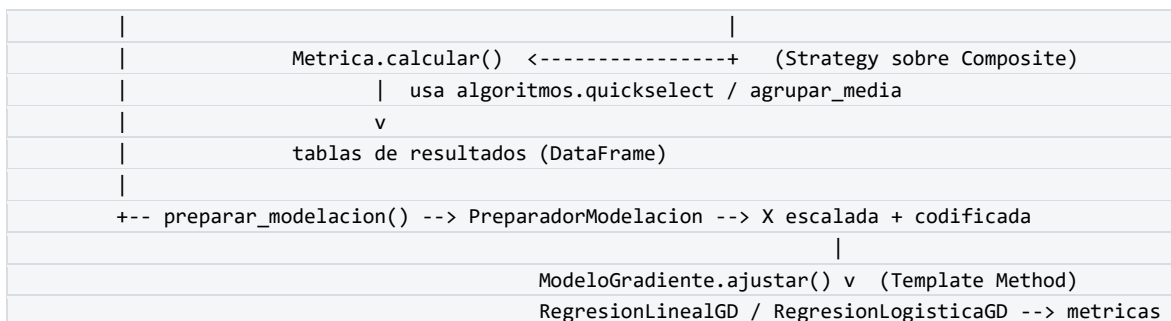
Tabla 3. Módulos del paquete `src/nucleo`

Módulo	Responsabilidad	Componentes principales
<code>algoritmos.py</code>	Algoritmos estructurados y recursivos con su complejidad declarada.	<code>agrupar_media</code> , <code>ordenar_merge</code> , <code>quickselect</code> , <code>mediana</code> , <code>busqueda_binaria</code> , <code>contar_en_rango</code>
<code>benchmark.py</code>	Mediciones reproducibles de tiempo y memoria; curvas de escalamiento.	<code>medir</code> , <code>comparar</code> , <code>curva_escalamiento</code> , <code>estimar_orden</code> , <code>Medicion</code>
<code>dominio.py</code>	Modelo de dominio con invariantes garantizados.	<code>Titulado</code> , <code>Cohorte</code>
<code>metricas.py</code>	Indicadores intercambiables sobre una cohorte.	<code>Metrica</code> (abstracta) y seis subclases
<code>jerarquia.py</code>	Árbol Sistema → Tipo → Institución → Carrera con agregación recursiva.	<code>NodoJerarquia</code> , <code>construir_jerarquia</code>
<code>preparacion.py</code>	Escalamiento, codificación y partición para modelar.	<code>Transformador</code> , <code>EscaladorEstandar</code> , <code>EscaladorMinMax</code> , <code>CodificadorOneHot</code> , <code>PreparadorModelacion</code>
<code>modelos.py</code>	Optimización por descenso de gradiente y solución cerrada.	<code>ModeloGradiente</code> , <code>RegresionLinealGD</code> , <code>RegresionLogisticaGD</code> , <code>RegresionLinealCerrada</code>
<code>fachada.py</code>	Punto de entrada único del núcleo.	<code>NucleoAnalitico</code> , <code>cargar_datos</code> , <code>ResultadoModelacion</code>
<code>validacion_f3.py</code>	Reglas de coherencia del núcleo, sobre el motor de la Fase 2.	<code>validador_nucleo</code>

4.2 Flujo de datos y dependencias

Flujo de datos del núcleo: cada flecha es una llamada con parámetros explícitos y retorno verificable

<code>dataset analitico F2 (DataFrame)</code>
v <code>fachada.cargar_datos()</code>
<code>NucleoAnalitico --> Cohorte.desde_dataframe() --> [Titulado, Titulado, ...]</code>
<code>construir_jerarquia()</code> v
+-----> <code>NodoJerarquia (arbol recursivo)</code>



Las dependencias apuntan en un solo sentido: fachada → {dominio, jerarquia, metricas, preparacion, modelos} → algoritmos. El módulo algoritmos.py no importa nada del proyecto; dominio.py importa solo algoritmos y config. Ese bajo acoplamiento es lo que permitió probar cada módulo por separado con datos contruidos en memoria, sin necesidad de los CSV del SIES.

4.3 Cohesión, acoplamiento y escalabilidad

- **Alta cohesión.** Cada módulo agrupa lo que cambia junto: los algoritmos no saben qué es un titulado; las métricas no saben cómo se construye una cohorte; los modelos no saben de dónde salen las matrices.
- **Bajo acoplamiento.** Los módulos se comunican por interfaces pequeñas (una cohorte iterable, una matriz NumPy, una función que devuelve un booleano y un detalle). Reemplazar la implementación de la mediana o agregar una métrica no toca al resto.
- **Escalabilidad.** Agregar un nivel a la jerarquía es cambiar una tupla de nombres; agregar un modelo es escribir dos ganchos; agregar una regla de validación es una línea. Ninguna de esas extensiones exige modificar código existente.

V. Codificación funcional: algoritmos estructurados y recursivos

Cada algoritmo se implementa sin dependencias externas, declara su complejidad en la docstring y se verifica contra una referencia independiente sobre los datos reales: la sobreduración de la cohorte completa, 1,25 millones de enteros con miles de valores repetidos. La corrección se establece antes de medir la eficiencia; medir un algoritmo incorrecto no tiene sentido.

5.1 Agregación en una pasada (programación estructurada)

src/nucleo/algoritmos.py — agregación $O(n)$ con estructuras de control básicas

```
def agrupar_media(claves, valores):
    """Media por clave en una sola pasada. Tiempo  $O(n)$ , espacio  $O(k)$ ."""
    if len(claves) != len(valores):
        raise ValueError("claves y valores deben tener el mismo largo")
    suma, cuenta = {}, {} # dos acumuladores por grupo
    for clave, valor in zip(claves, valores): # una unica pasada sobre los datos
        suma[clave] = suma.get(clave, 0.0) + valor
        cuenta[clave] = cuenta.get(clave, 0) + 1
    return {clave: suma[clave] / cuenta[clave] for clave in suma}
```

Verificación: coincide exactamente con `groupby(...).mean()` de pandas en las diez áreas de conocimiento. Se conserva además una versión ingenua $O(n \cdot k)$, que recorre toda la secuencia una vez por grupo, como término de comparación en el análisis de eficiencia.

5.2 Merge sort recursivo

Divide y vencerás: cada llamada resuelve la mitad del problema

```
def ordenar_merge(datos):
    """Merge sort. Tiempo  $O(n \log n)$ , espacio  $O(n)$ , profundidad  $O(\log n)$ . Estable."""
    n = len(datos)
    if n <= 1: # caso base: ya esta ordenado
        return list(datos)
    mitad = n // 2
    izquierda = ordenar_merge(datos[:mitad]) # divide ...
    derecha = ordenar_merge(datos[mitad:])
    return _mezclar(izquierda, derecha) # ... y venceras: mezcla en  $O(n)$ 
```

Funciona con cualquier tipo que implemente el operador `<`, lo que permite ordenar objetos Titulado directamente (sección VII). Con 1,25 millones de elementos la profundidad de recursión es de 21 niveles, muy por debajo del límite de Python. Verificación: idéntico a `sorted()` y estable.

5.3 Quickselect: la mediana sin ordenar

Selección recursiva del k-ésimo menor sin ordenar la secuencia completa

```
def _quickselect(datos, k, rng):
    if len(datos) == 1:
        # caso base
        return datos[0]
    pivote = datos[rng.randrange(len(datos))] # pivote aleatorio: O(n) esperado
    menores = [x for x in datos if x < pivote]
    iguales = [x for x in datos if x == pivote]
    mayores = [x for x in datos if x > pivote]
    if k < len(menores):
        # recurre solo en la particion que
        return _quickselect(menores, k, rng) # contiene la posicion buscada
    if k < len(menores) + len(iguales):
        return pivote
    return _quickselect(mayores, k - len(menores) - len(iguales), rng)
```

Verificación: mediana y percentil 75 idénticos a los de NumPy. La partición en tres listas (menores, iguales, mayores) es deliberada: la sobreduración tiene miles de valores repetidos, y una partición binaria degradaría el algoritmo ante los empates.

5.4 Búsqueda binaria recursiva y conteo por umbral

Sobre una secuencia ordenada, dos búsquedas binarias recursivas (cota inferior y cota superior) permiten contar cuántos elementos caen en un intervalo en $O(\log n)$, sin recorrer los valores. Con ellas se reconstruyen las cuatro categorías de rezago de la Fase 2 — oportuna, leve, moderado, severo— y el resultado coincide exactamente con `value_counts` sobre el dataset completo. La cota superior es necesaria: avanzar linealmente sobre los valores iguales al umbral degradaría el conteo a $O(n)$, porque un mismo valor de sobreduración se repite decenas de miles de veces.

VI. Análisis de complejidad y eficiencia

6.1 Cotas teóricas

Tabla 4. Complejidad temporal y espacial de los algoritmos del núcleo

Algoritmo	Tiempo	Espacio adicional	Recursión
agrupar_media	$O(n)$	$O(k)$	no
agrupar_media_ingenua	$O(n \cdot k)$	$O(n)$	no
ordenar_insercion	$O(n^2)$	$O(1)$	no
ordenar_merge	$O(n \log n)$	$O(n)$	profundidad $O(\log n)$
sorted / numpy.sort (C)	$O(n \log n)$	$O(n)$	—
quickselect / mediana	$O(n)$ esperado, $O(n^2)$ peor caso	$O(n)$	profundidad $O(\log n)$ esperada
mediana_ordenando	$O(n \log n)$	$O(n)$	no
busqueda_lineal	$O(n)$	$O(1)$	no
busqueda_binaria	$O(\log n)$	$O(1)$ por nivel	profundidad $O(\log n)$
NodoJerarquia.total (memorizado)	$O(1)$ amortizado	$O(\text{nodos})$	recursivo con cache

6.2 Método de medición

Las mediciones usan `timeit.repeat` y reportan el mínimo de las repeticiones: el ruido del sistema operativo solo puede sumar tiempo, por lo que el mínimo es el mejor estimador del costo intrínseco del algoritmo (Python Software Foundation, 2025a). La memoria se mide con `tracemalloc` como pico de asignación durante la llamada (Python Software Foundation, 2025b). Además de medir un tamaño fijo, se mide cada algoritmo para tamaños crecientes y se ajusta una recta en escala log-log: la pendiente es el exponente empírico de crecimiento, que debe coincidir con la cota teórica (≈ 1 lineal, $\approx 1,1$ para $n \log n$ en este rango, ≈ 2 cuadrático). Este contraste entre cota y medición es la parte central del análisis (Cormen et al., 2022).

6.3 Comparación de implementaciones a tamaño fijo

Con $n = 200.000$ elementos, cada operación se compara entre la implementación propia, su versión ingenua y la de `pandas` o `NumPy`. La columna razón indica cuántas veces más lenta es cada implementación respecto de la mejor.

Tabla 5. Tiempo y memoria de cada implementación con 200.000 elementos

Operación	Implementación	Tiempo	Memoria pico (KB)	Razón vs. mejor
agregación	agrupar_media $O(n)$	0,0449 s	0,7	5,5×
agregación	agrupar_media_ingenua $O(n \cdot k)$	0,0847 s	603,9	10,4×
agregación	pandas groupby	0,0081 s	3132,6	1,0×
ordenamiento	numpy.sort	0,0087 s	3127,8	1,0×
ordenamiento	sorted (Timsort en C)	0,0121 s	2343,6	1,4×
ordenamiento	ordenar_merge $O(n \log n)$	0,5762 s	3250,9	66,4×
mediana	numpy.median	0,0094 s	3128,2	1,0×
mediana	mediana_ordenando $O(n \log n)$	0,0122 s	2343,6	1,3×
mediana	quickselect $O(n)$	0,0538 s	4422,2	5,7×
búsqueda	busqueda_binaria $O(\log n)$	1,0 μ s	0,0	1,0×
búsqueda	busqueda_lineal $O(n)$	0,0065 s	0,1	6516,0×

La inserción $O(n^2)$ se midió aparte con $n = 3.000$: con 200.000 elementos tardaría horas.

6.4 Curvas de escalamiento y exponente empírico

Tabla 6. Exponente empírico de crecimiento frente a la cota teórica

Algoritmo	Cota teórica	Exponente empírico	Rango de n medido
agrupar_media	$O(n)$	1,00	20.000 – 640.000
ordenar_merge	$O(n \log n)$	1,08	20.000 – 640.000
quickselect	$O(n)$	1,22	20.000 – 640.000
sorted	(C)	1,02	20.000 – 640.000
ordenar_insercion	$O(n^2)$	2,04	500 – 4.000

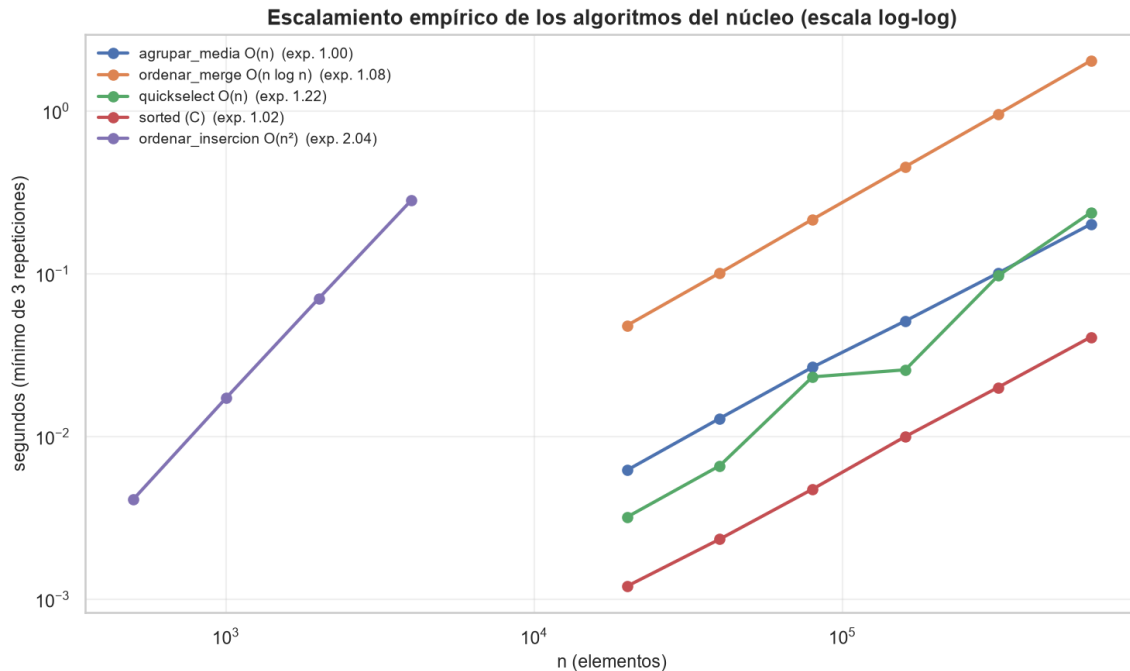


Figura 1. Escalamiento empírico de los algoritmos del núcleo en escala log-log; la pendiente de cada recta es el exponente de crecimiento.

6.5 Memoria y memorización

Tabla 7. Pico de memoria por algoritmo ($n = 200.000$)

Algoritmo	Espacio teórico	Pico (KB)	KB por 1.000 elementos
ordenar_merge	$O(n)$	3253,5	16,27
quickselect (mediana)	$O(n)$	4422,0	22,11
sorted	$O(n)$	2343,6	11,72
agrupar_media	$O(k)$	0,7	0,00
busqueda_binaria	$O(\log n)$ pila	0,0	0,00

En el árbol jerárquico, el total de titulados de un nodo se calcula recursivamente sumando el de sus hijos. Sin memorización, cada consulta recorre el subárbol completo; con ella, cada nodo recuerda su total hasta que una inserción lo invalida. Consultar el total de todas las instituciones del nivel intermedio tarda 9,0 μ s con cache frente a 0,0019 s sin ella: 209,6 veces más rápido. Es el principio de `functools.lru_cache` aplicado a una estructura mutable.

6.6 Interpretación y elección justificada

1. La cota teórica se confirma empíricamente. Los exponentes ajustados son 1,00 para la agregación ($O(n)$), 1,02 para sorted y 1,08 para merge sort (el factor $\log n$ aporta

ese exceso sobre 1 en el rango medido) y 2,04 para inserción ($O(n^2)$). Las implementaciones se comportan como su análisis predice.

2. Quickselect es $O(n)$ en comparaciones, pero su exponente empírico (1,22) queda por encima de 1. La causa está en la implementación: cada nivel de recursión construye tres listas nuevas, de modo que el costo de asignar memoria crece con n y se suma al de comparar. Aun así, a igual n es diez veces más rápido que merge sort. Es la demostración de que el análisis asintótico debe complementarse con la medición: la cota describe el crecimiento, no la constante ni el comportamiento de la memoria.
3. El orden de crecimiento gana entre clases distintas; dentro de la misma clase decide la constante. La búsqueda binaria es más de seis mil veces más rápida que la lineal: $O(\log n)$ contra $O(n)$. En cambio, sorted y numpy.sort comparten el $O(n \log n)$ de merge sort y son entre 50 y 65 veces más rápidos porque están escritos en C. Por eso mediana_ordenando, que ordena en C, supera a quickselect en Python puro pese a su peor cota.
4. Elección final. En producción se conservan pandas y NumPy para las operaciones masivas, porque a igual complejidad su constante es uno o dos órdenes de magnitud menor y su memoria es comparable. Los algoritmos propios se usan donde su orden es estrictamente mejor y no existe un equivalente en C sobre objetos: búsqueda binaria para conteos por umbral, quickselect para estadísticos de orden sobre listas de Titulado, y la memorización del árbol jerárquico. Y cumplen el rol formativo central: saber por qué pandas es rápido es lo que permite usarlo bien.

VII. Programación orientada a objetos y patrones de diseño

Las clases del núcleo se diseñaron a partir de un principio cada una, y no al revés: primero la necesidad, después la clase. Encapsulamiento donde hay invariantes que proteger; herencia donde hay comportamiento común real; composición donde la relación es «tiene un» y no «es un». Los patrones se nombran según el catálogo de Gamma et al. (1994) y se justifica su función en la arquitectura frente a la alternativa sin patrón.

7.1 Encapsulamiento: Titulado

src/nucleo/dominio.py — atributos privados, validación en setters y propiedades derivadas

```
class Titulado:
    __slots__ = ("_mrun", "_duracion_teorica", "_duracion_real", ...) # sin __dict__

    @property
    def duracion_real(self) -> int:
        return self._duracion_real

    @duracion_real.setter
    def duracion_real(self, valor: int) -> None:
        if int(valor) < config.DURACION_REAL_MIN: # el invariante vive en el objeto
            raise ValueError("la duracion real debe ser >= 1")
        self._duracion_real = int(valor)

    @property
    def sobreduracion(self) -> int: # derivada: nunca inconsistente
        return self._duracion_real - self._duracion_teorica

    def __lt__(self, otro): # orden natural por sobreduracion
        return self.sobreduracion < otro.sobreduracion
```

Los atributos de identidad son de solo lectura; las duraciones pasan por setters que validan; la sobreduración, el índice y la categoría de rezago son propiedades calculadas que no pueden fijarse a mano. Un objeto Titulado no puede quedar en estado incoherente. `__slots__` elimina el diccionario por instancia y reduce a la mitad la memoria, lo que importa al instanciar 300.000 objetos (1,8 segundos, 6 microsegundos por objeto). Los métodos especiales `__eq__`, `__hash__` y `__lt__` integran la clase con el lenguaje: se puede usar en conjuntos, comparar y ordenar con `sorted` o con el `merge sort` propio (Ramalho, 2022).

7.2 Protocolo de secuencia: Cohorte

Cohorte implementa `__len__`, `__iter__`, `__getitem__` y `__contains__`, por lo que se comporta como una lista de Python. Las operaciones `filtrar` y `agrupar_por` devuelven cohortes nuevas

sin modificar la original, en línea con la convención del proyecto. La cohorte no calcula indicadores por sí misma: delega en objetos *Metrica*.

7.3 Herencia y polimorfismo: *Metrica* (patrón Strategy)

src/nucleo/metricas.py — clase abstracta, subclasses intercambiables

```
class Metrica(ABC):
    nombre: str = "metrica"

    @abstractmethod
    def calcular(self, cohorte) -> float: ...      # cada subclase aporta la formula

class MedianaSobreduracion(Metrica):
    nombre = "sobreduracion_mediana"
    def calcular(self, cohorte):
        self._exigir_no_vacia(cohorte)            # comportamiento heredado
        return float(mediana(cohorte.valores("sobreduracion"))) # quickselect O(n)

# Cohorte no conoce las subclasses: acepta cualquier Metrica (polimorfismo)
cohorte.calcular(MedianaSobreduracion())
cohorte.calcular(PercentilSobreduracion(75))
```

Metrica es una clase abstracta (módulo *abc*; Python Software Foundation, 2025c): no puede instanciarse y obliga a cada subclase a implementar *calcular*. *Cohorte.calcular* acepta cualquier subclase sin conocerla. Esto es el patrón Strategy: el algoritmo de cálculo es un objeto intercambiable. Su función en la arquitectura es que agregar un indicador nuevo no exige modificar *Cohorte*, *NodoJerarquia* ni la fachada; la alternativa —un método *calcular(tipo)* con un *if/elif* por métrica— obliga a editar la clase con cada indicador.

7.4 Recursión sobre objetos: *NodoJerarquia* (patrón Composite)

El dominio tiene una jerarquía natural: sistema → tipo de institución → institución → carrera. Se modela como un árbol donde cada nodo tiene la misma interfaz sea hoja o nodo interno, de modo que *total()*, *calcular(metrica)* o *profundidad()* funcionan igual en cualquier nivel y se resuelven recursivamente sobre los hijos. La inserción también es recursiva: el caso base es la ruta vacía; el caso recursivo obtiene o crea el hijo del primer nivel y delega el resto.

src/nucleo/jerarquia.py — inserción, total memorizado y descenso recursivos

```
def insertar(self, titulado, ruta, niveles):
    self._invalidar_cache()          # memorizacion: el total cambio
    if not ruta:                     # caso base: hoja
        self._titulados.append(titulado)
        return
    hijo = self._hijos.setdefault(ruta[0], NodoJerarquia(ruta[0], niveles[0], padre=self))
```

```

    hijo.insertar(titulado, ruta[1:], niveles[1:]) # caso recursivo

def total(self) -> int: # recursivo y memorizado
    if self._cache_total is None:
        self._cache_total = len(self._titulados) + sum(h.total() for h in self._hijos.values())
    return self._cache_total

def descender_por_maximo(self, metrica, minimo=30): # 'donde se concentra el rezago'
    candidatos = [h for h in self._hijos.values() if h.total() >= minimo]
    if not candidatos:
        return [self]
    mejor = max(candidatos, key=lambda h: h.calcular(metrica))
    return [self] + mejor.descender_por_maximo(metrica, minimo)

```

Tabla 8. Sobreduración media por tipo de institución obtenida del árbol (300.000 titulados)

Nivel 1	Titulados	Sobreduración media	Titulación oportuna
Universidades	137.436	3,32 sem.	12,8 %
Centros de Formación Técnica	51.515	2,83 sem.	14,0 %
Institutos Profesionales	111.049	2,06 sem.	28,8 %

Coincide con lo obtenido por pandas en la Fase 2 (3,32 / 2,83 / 2,05): la regla `metrica_composite` del validador lo comprueba con tolerancia $1e-9$.

7.5 Herencia con propósito: Transformador y ModeloGradiente

Dos jerarquías más completan el núcleo. Transformador (abstracta) aporta la validación de entrada, el control de «ajustado antes de transformar» y `ajustar_transformar`; EscaladorEstandar y EscaladorMinMax heredan todo eso y sobreescriben solo los ganchos `_aprender` y `_aplicar`. ModeloGradiente (abstracta) contiene el bucle completo de optimización y deja como ganchos la función de enlace y la pérdida; RegresionLinealGD y RegresionLogisticaGD los definen en cuatro líneas cada una. Este segundo caso es el patrón Template Method: el esqueleto del algoritmo se escribe una vez y las subclases rellenan los pasos variables. RegresionLinealCerrada comparte la interfaz pública sin heredar —polimorfismo por interfaz—, lo que basta para que el notebook trate a todos los modelos por igual.

7.6 Patrones de diseño y su función en la arquitectura

Tabla 9. Patrones aplicados, función y alternativa descartada

Patrón	Dónde	Función en la arquitectura	Alternativa descartada
Strategy	Metrica y subclases	Indicadores intercambiables sin tocar Cohorte ni la jerarquía.	Un if/elif por métrica dentro de Cohorte.

Patrón	Dónde	Función en la arquitectura	Alternativa descartada
Composite	NodoJerarquia	Misma interfaz en hojas y nodos internos; agregación recursiva uniforme a cualquier nivel.	Un groupby distinto por nivel: tres códigos para la misma pregunta.
Template Method	ModeloGradiente.ajustar	El bucle de optimización se escribe una vez; cada modelo aporta enlace y pérdida.	Copiar el bucle en cada modelo: duplicación y divergencia futura.
Facade	NucleoAnalitico	El notebook no depende de los detalles internos de nueve módulos.	Que el notebook importe y coordine cada módulo: alto acoplamiento.
Composición	PreparadorModelacion	Tiene un escalador y un codificador; los coordina sin heredar de ellos.	Herencia múltiple: relación «es un» incorrecta.

7.7 Diagrama de clases

Diagrama de clases del núcleo (* = método abstracto; <> = composición)

<<abstract>>				<<abstract>>			
Metrica				Transformador			
+ nombre				+ ajustar(X) + transformar(X)			
+ calcular(cohorte)*				# _aprender(X)* # _aplicar(X)*			
^				^			
+-----+-----+-----+-----+				+-----+-----+			
MediaSobre	Mediana	Percentil	TasaOportuna ...	EscaladorEstandar	EscaladorMinMax		
Titulado Cohorte <>--- 0..* Titulado				NodoJerarquia <>--- 0..* NodoJerarquia			
- _mrun, _duracion_*		+ __len__/_iter__/_getitem__		+ insertar(t, ruta)		(recursivo)	
+ sobreduracion (prop)		+ filtrar(pred) -> Cohorte		+ total()		(memorizado)	
+ es_oportuna (prop)		+ agrupar_por(attr)		+ calcular(metrica)			
(Composite+Strategy)							
+ __lt__, __eq__		+ calcular(metrica) -> float		+ descender_por_maximo()			
<<abstract>>							
ModeloGradiente				RegresionLinealCerrada (misma interfaz, sin herencia)			
+ ajustar(X, y) <- bucle fijo (Template Method)							
# _enlace(z)* # _perdida(y, p)*							
^							
+-----+-----+							
RegresionLinealGD		RegresionLogisticaGD					
NucleoAnalitico (Facade) --uses--> Cohorte, NodoJerarquia, Metrica, PreparadorModelacion, ModeloGradiente							

VIII. Preprocesamiento y transformación para la modelación

El dataset de la Fase 2 está limpio y tipado. Modelar exige dos transformaciones adicionales, y ambas se implementan como clases del núcleo con una interfaz común, validación de entrada y separación estricta entre ajustar y transformar.

Tabla 10. Transformaciones aplicadas a la matriz de modelación y su justificación

Transformación	Variables	Justificación técnica
Estandarización ($z = (x - \text{media}) / \text{desviación}$)	dur_total_carr, edad_ingreso	El descenso de gradiente converge mal si las variables tienen escalas distintas (1–24 semestres frente a 15–80 años). Se estandariza a media 0 y desviación 1 con parámetros calculados solo en el conjunto de entrenamiento, para no filtrar información al de prueba. Una columna constante (desviación 0) se deja en 0 en lugar de dividir por cero.
Codificación one-hot	genero, tipo_inst_1, area_conocimiento, modalidad	Las categóricas se convierten en indicadoras 0/1. Se omite una categoría por variable (la de referencia) para evitar la colinealidad perfecta con el intercepto. Una categoría no vista en entrenamiento produce ceros, no un error.
Partición entrenamiento/prueba	200.000 registros → 160.000 / 40.000	Permutación aleatoria con semilla fija (reproducible); particiones disjuntas. Las métricas se reportan en prueba.

8.1 Validaciones del resultado

Sobre la matriz resultante se comprueba: ausencia de NaN; media ≈ 0 y desviación ≈ 1 en las numéricas del conjunto de entrenamiento; indicadoras con valores exclusivamente 0/1; mismo número de columnas (16) en entrenamiento y prueba; particiones disjuntas y completas; y que la media de las numéricas de prueba no sea exactamente 0, prueba de que se escaló con parámetros de entrenamiento y no propios. Las siete comprobaciones se ejecutan como aserciones en el notebook (sección 6) y quedan reflejadas en la regla `matrices_modelacion` del validador.

IX. Modelación desde primeros principios

9.1 Descenso de gradiente

El descenso de gradiente es un algoritmo de optimización iterativo: en cada paso calcula la predicción, el residuo y el gradiente de la pérdida, y mueve los parámetros en la dirección de máximo descenso. Complejidad $O(\text{iteraciones} \cdot n \cdot p)$ en tiempo y $O(n \cdot p)$ en espacio. Para verificar que la implementación converge al óptimo, la regresión lineal se contrasta con su solución cerrada por ecuaciones normales, que es exacta.

src/nucleo/modelos.py — bucle de optimización con detección de divergencia

```
def ajustar(self, X, y):                                # Template Method: fijo para toda subclase
    X, y = _validar_xy(X, y)
    n, p = X.shape
    self._pesos, self._sesgo, self._historial = np.zeros(p), 0.0, []
    for _ in range(self._iteraciones):
        predicción = self._enlace(X @ self._pesos + self._sesgo)    # gancho 1
        residuo = predicción - y
        self._pesos -= self._tasa * (X.T @ residuo) / n              # paso de gradiente
        self._sesgo -= self._tasa * residuo.mean()
        pérdida = self._perdida(y, predicción)                      # gancho 2
        if not np.isfinite(pérdida) or pérdida > 1e6 * self._historial[0]:
            raise RuntimeError("el descenso de gradiente divergió; reduzca la tasa")
        self._historial.append(pérdida)
    return self
```

9.2 Convergencia verificada

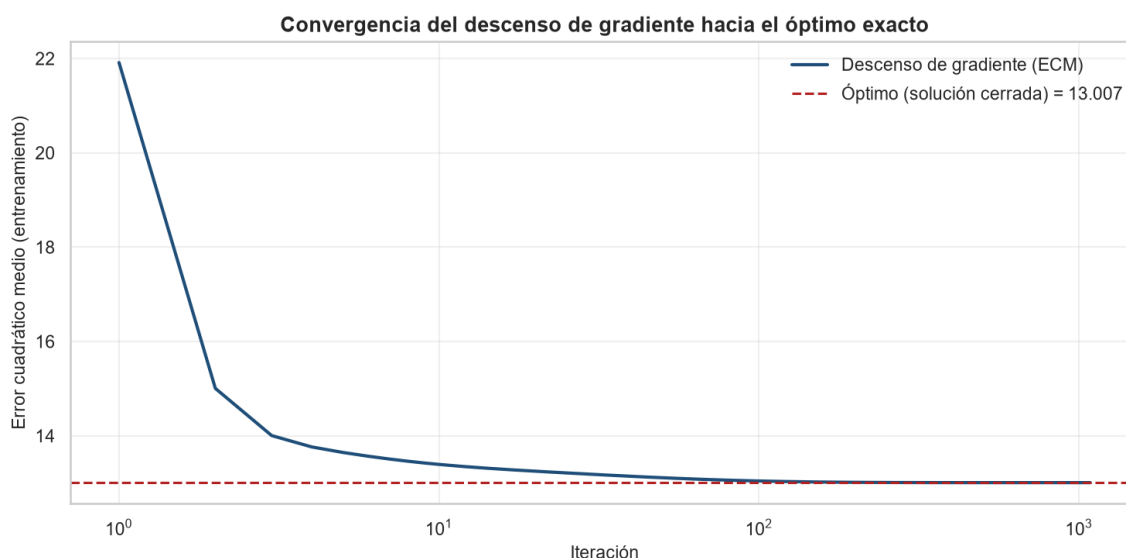


Figura 2. Error cuadrático medio por iteración del descenso de gradiente frente al óptimo exacto de la solución cerrada.

Tabla 11. Coeficientes de la regresión lineal: descenso de gradiente frente a solución cerrada (variables estandarizadas)

Variable	Descenso de gradiente	Solución cerrada	Diferencia	Logística
dur_total_carr	-0,382	-0,382	0,000	0,055
edad_ingreso	-0,525	-0,525	0,000	0,291
genero=Mujer	-0,405	-0,405	0,000	0,128
tipo_inst_1=Institutos Profesionales	-0,544	-0,544	0,000	0,889
tipo_inst_1=Universidades	0,916	0,916	0,000	-0,040
area_conocimiento=Agropecuaria	0,712	0,712	0,000	-0,377
area_conocimiento=Arte y Arquitectura	0,002	0,002	0,000	0,109
area_conocimiento=Ciencias Básicas	1,358	1,364	0,006	-0,423
area_conocimiento=Ciencias Sociales	-0,522	-0,522	0,000	0,159
area_conocimiento=Derecho	2,862	2,862	0,000	-0,475
area_conocimiento=Educación	-0,802	-0,802	0,000	0,329
area_conocimiento=Humanidades	-0,543	-0,554	0,012	0,149
area_conocimiento=Salud	-0,377	-0,377	0,000	0,452
area_conocimiento=Tecnología	0,906	0,906	0,000	-0,304
modalidad=Presencial	-0,145	-0,145	0,000	-0,250
modalidad=Semipresencial	0,017	0,017	0,001	0,103

Diferencia máxima 0,012, en la categoría menos frecuente (Humanidades), cuyo gradiente es el más pequeño y por tanto el más lento en converger. Categorías de referencia: Hombre, Centros de Formación Técnica, Administración y Comercio, No Presencial.

El ensayo de tasas de aprendizaje muestra las dos caras del algoritmo: con 0,05 a 0,5 converge (más rápido cuanto mayor la tasa); con 0,9 o más oscila y diverge, y el modelo lo detecta lanzando una excepción explícita en lugar de devolver pesos infinitos o NaN. Es un caso de excepción controlada que la suite de pruebas verifica.

9.3 Desempeño e interpretación

Tabla 12. Desempeño de los modelos en el conjunto de prueba (40.000 registros)

Modelo	RMSE	R ²	Exactitud	Precisión	Sensibilidad	F1
Lineal (gradiente)	3,590	0,085	—	—	—	—
Lineal (cerrada)	3,590	0,085	—	—	—	—
Logística (umbral 0,5)	—	—	0,812	0,450	0,040	0,074
Línea base (clase mayoritaria)	—	—	0,813	0	0	0

Tabla 13. Efecto del umbral de decisión en la regresión logística (prueba)

Umbral	Exactitud	Precisión	Sensibilidad	F1
0,50	0,812	0,450	0,040	0,074
0,40	0,803	0,404	0,119	0,184
0,30	0,773	0,374	0,320	0,345
0,19	0,676	0,309	0,596	0,407

- **Capacidad explicativa limitada, como se anticipó en F2.** Un R^2 de 0,085 indica que las variables institucionales y demográficas disponibles explican una fracción pequeña de la variación individual: el rezago depende sobre todo de trayectorias personales que la base no registra. Los coeficientes, sin embargo, son coherentes con el análisis descriptivo: estudiar en una universidad suma 0,92 desviaciones respecto de un CFT, Derecho suma 2,86, ser mujer resta 0,41, y una mayor duración teórica o edad de ingreso restan.
- **En clasificación, la exactitud engaña.** Con un 19 % de titulación oportuna, predecir siempre «no oportuna» acierta el 81,3 %; la logística con umbral 0,5 llega al 81,2 % porque casi nunca predice la clase minoritaria (sensibilidad 4 %). Bajar el umbral a la prevalencia (0,19) eleva la sensibilidad al 60 % a costa de la precisión. La métrica correcta depende del uso: detectar riesgo de rezago exige sensibilidad.
- **Proyección a F4.** Variables adicionales (cohorte de ingreso, región, jornada), términos de interacción y modelos no lineales son los candidatos para mejorar el ajuste; el núcleo de esta fase los admite sin cambios estructurales.

X. Validación técnica y verificación del código

La verificación se realiza en cuatro niveles con evidencia registrada en el repositorio.

10.1 Corrección contra referencias independientes

Cada algoritmo se contrasta, sobre los datos reales, con una referencia externa: la agregación con pandas, el ordenamiento con sorted, la mediana y el percentil con NumPy, el conteo por umbral con value_counts, y las métricas sobre objetos con las mismas métricas de pandas sobre las mismas filas. Todas las comparaciones son aserciones del notebook: una discrepancia detiene la ejecución.

10.2 Pruebas automatizadas

Tabla 14. Cobertura de la suite de pruebas del núcleo (tests/test_nucleo.py)

Módulo	Casos normales	Casos límite	Excepciones
algoritmos	Coincidencia con pandas, sorted, statistics; estabilidad; no mutación	Secuencia vacía y de un elemento; bordes de cotas con valores repetidos	Largos distintos; k fuera de rango; mediana de vacío
benchmark	Exponente ≈ 1 para lineal, ≈ 2 para cuadrático; orden por tiempo	—	Menos de dos tamaños; tamaños iguales
dominio	Propiedades derivadas; orden natural; igualdad; construcción desde dict y Series	Cambio de duración recalcula derivadas	Identidad de solo lectura; setters inválidos; tipo ajeno en Cohorte
metricas	Polimorfismo sobre cohorte; resumen estándar	—	Cohorte vacía; percentil fuera de [0, 100]
jerarquia	Totales recursivos; profundidad; hojas; descenso por máximo; DataFrame	Nodo vacío; cache invalidada tras insertar	Búsqueda de nodo inexistente
preparacion	Media 0 / desviación 1; inversión; rango [0, 1]; partición reproducible	Columna constante; categoría no vista	Transformar sin ajustar; columnas distintas; NaN; proporción inválida
modelos	Convergencia GD $\approx$ cerrada; pérdida decreciente; logística en datos separables	Pesos devueltos como copia	Predecir sin ajustar; formas incompatibles; tasa cero; divergencia

Resultado de pytest tests/ -v

```
===== test session starts =====
===== 73 passed in 3.18s =====
(salida completa en el Anexo B)
```

10.3 Reglas de coherencia con la Fase 2

El motor de reglas de la Fase 2 (Regla, ValidadorDataset) se reutiliza sin modificaciones sobre los artefactos de esta fase; es un caso de reutilización entre fases que la arquitectura modular hizo trivial.

Tabla 15. Reglas de validación del núcleo y resultado

Regla	Comprobación	Estado
cohortes_y_arbol	La jerarquía contiene exactamente los titulados de la cohorte.	PASA
hojas_suman_total	Las hojas del arbol suman el total de la raíz (recursion correcta).	PASA
metrica_composite	Una metrica via el arbol coincide con la misma via la cohorte.	PASA
tasa_oportuna_coherente	La tasa de titulacion oportuna sobre objetos coincide con la de F2.	PASA
convergencia_gd	El descenso de gradiente converge a la solucion cerrada.	PASA
perdida_decrece	La perdida final es menor que la inicial.	PASA
matrices_modelacion	Las matrices no tienen NaN y comparten columnas.	PASA
metricas_finitas	Todas las metricas de desempeno son numeros finitos.	PASA
r2_positivo	El modelo lineal explica varianza en el conjunto de prueba.	PASA

10.4 Ejecución de extremo a extremo

El notebook se ejecuta de corrido con nbconvert en el entorno declarado: 27 celdas de código numeradas de 1 a 27, sin errores ni advertencias que interrumpan el flujo. Es el mismo procedimiento con que se verificaron los notebooks de las fases anteriores.

XI. Notebook F3_Nucleo_Algoritmico.ipynb

El notebook de la fase tiene 57 celdas —30 de documentación en markdown y 27 de código— ordenadas según el flujo que la rúbrica establece: mejoras de código, complejidad y programación orientada a objetos, seguidas de preprocesamiento, modelación y validación. Cada bloque abre con una explicación de qué se hace y por qué, y cierra con un resultado intermedio verificable.

Tabla 16. Estructura del notebook y su correspondencia con el informe

Sección del notebook	Contenido	Sección del informe
1. Entorno, datos y mejoras	Versiones, origen de datos, tabla de mejoras integradas desde F2	III
2. Arquitectura	Inventario por introspección y flujo de datos	IV
3. Algoritmos	Cuatro algoritmos verificados contra referencias	V
4. Complejidad y eficiencia	Cotas, comparación a n fijo, curvas, memoria, interpretación	VI
5. POO	Encapsulamiento, secuencia, Strategy, Composite, memorización, diagrama	VII
6. Preprocesamiento	Escalamiento, codificación, partición y siete validaciones	VIII
7. Modelación	Convergencia, ensayo de tasas, desempeño, umbrales, interpretación	IX
8. Validación	pytest robusto y nueve reglas de coherencia	X
9. Síntesis	Balance y proyección a F4	XIII

Trazabilidad: las diez tablas del notebook se exportan a reports/tables/f3_*.csv y las dos figuras a reports/figures/f3_*.png; todas las cifras de este informe provienen de esos archivos.

XII. Repositorio GitHub

12.1 Estructura de la Fase 3

Carpeta F3 diferenciada de F1–F2 y código del núcleo en su propio subpaquete

proyecto-titulados/		
-- F1/	F1_Definición.ipynb	(Fase 1, entregada)
-- F2/	F2_1 · F2_2 · F2_3 .ipynb	(Fase 2, entregada)
-- F3/		
	-- F3_Nucleo_Algoritmico.ipynb	(Fase 3: 57 celdas, ejecutado de corrido)
-- src/		
	-- config, ingesta, limpieza, transformacion, validacion, viz, pipeline	(F1-F2)
	-- nucleo/	(F3)
	-- algoritmos.py benchmark.py dominio.py metricas.py jerarquia.py	
	-- preparacion.py modelos.py fachada.py validacion_f3.py	
-- tests/		
	-- test_pipeline.py	(22 pruebas, F2)
	-- test_nucleo.py	(51 pruebas, F3)
-- reports/	figures/f3_*.png tables/f3_*.csv	
-- informe/	Sumativa1 (entregado) · evidencias_f3/	
-- .gitignore	excluye los datos crudos y el entorno virtual	
-- README.md	con las instrucciones de ejecución del notebook F3	

12.2 Historial de commits de la fase

La Fase 3 se desarrolla en la rama fase-3/nucleo-algoritmico, creada a partir de main tal como fue entregada la Sumativa 1. Los commits siguen el orden de construcción real — algoritmos y mediciones, dominio orientado a objetos, modelos y fachada, pruebas, notebook— y sus mensajes explican qué se hizo y por qué.

Tabla 17. Commits del repositorio (31 en total; los de la Fase 3 encabezan la lista)

Hash	Fecha	Mensaje
b769379	2026-09-19	docs(f3): agrega el PDF del informe con la fecha de entrega
d10b470	2026-09-19	docs(f3): Primer Informe F3
6d348f8	2026-09-19	docs(f3): informe tecnico de la Fase 3 y evidencias de ejecucion
b8770f5	2026-09-19	docs(f3): notebook del nucleo algoritmico ejecutado de corrido
d6df25e	2026-09-19	test(nucleo): 51 pruebas de casos normales, limite y excepciones
40be4d9	2026-09-19	feat(nucleo): preparacion, modelos por descenso de gradiente y fachada
a1342a5	2026-09-19	feat(nucleo): modelo de dominio orientado a objetos
8d87723	2026-09-19	feat(nucleo): algoritmos estructurados y recursivos con analisis de comp...
76867c7	2026-09-15	chore: prepara la rama de la Fase 3 y sanea el repositorio
807ad02	2026-09-09	Reejecucion de notebook en el equipo de trabajo con documento pdf
9d8c578	2026-09-09	Reejecucion de notebook en el equipo de trabajo con Jupyter LAB

git log --oneline --graph

* b769379	docs(f3): agrega el PDF del informe con la fecha de entrega
* d10b470	docs(f3): Primer Informe F3
* 6d348f8	docs(f3): informe tecnico de la Fase 3 y evidencias de ejecucion
* b8770f5	docs(f3): notebook del nucleo algoritmico ejecutado de corrido
* d6df25e	test(nucleo): 51 pruebas de casos normales, limite y excepciones
* 40be4d9	feat(nucleo): preparacion, modelos por descenso de gradiente y fachada
* a1342a5	feat(nucleo): modelo de dominio orientado a objetos
* 8d87723	feat(nucleo): algoritmos estructurados y recursivos con analisis de complejidad
* 76867c7	chore: prepara la rama de la Fase 3 y sanea el repositorio
* 807ad02	Reejecucion de notebook en el equipo de trabajo con documento pdf
* 9d8c578	Reejecucion de notebook en el equipo de trabajo con Jupyter LAB
* 75b8d4b	Reejecucion de notebook en el equipo de trabajo
* 5d76ade	docs: actualiza el informe y el notebook F1 tras corregir la autoria

Contribuciones: el proyecto es individual; los commits están atribuidos a la cuenta institucional del autor (git shortlog en el Anexo C).

XIII. Conclusiones y proyección

13.1 Sobre el diseño algorítmico

Implementar los algoritmos desde cero y medirlos con rigor produjo un resultado que ninguna lectura teórica habría dado por sí sola: los exponentes empíricos confirman las cotas (1,00, 1,08, 2,04), pero también muestran dónde la cota no cuenta toda la historia — quickselect, $O(n)$ en comparaciones, escala como $n^{1,22}$ por el costo de asignar memoria— y por qué, a igual orden, una implementación en C supera a la mejor implementación en Python por dos órdenes de magnitud. La conclusión de ingeniería es concreta: pandas y NumPy para lo masivo; algoritmos propios donde su orden es estrictamente mejor y no hay equivalente sobre objetos.

13.2 Sobre la programación orientada a objetos

Cada clase del núcleo responde a un principio identificable: encapsulamiento donde hay invariantes (Titulado), herencia donde hay comportamiento común real (Transformador, ModeloGradiente), polimorfismo donde hay que intercambiar algoritmos (Metrica), composición donde la relación es de pertenencia (PreparadorModelacion), recursión donde la estructura es jerárquica (NodoJerarquia). Los cinco patrones aplicados no son decorativos: en cada caso se documenta la alternativa sin patrón y por qué es peor. La arquitectura resultante permitió probar cada módulo aislado con 51 pruebas que no necesitan los datos reales, y reutilizar el motor de validación de la Fase 2 sin modificarlo.

13.3 Sobre la modelación

El descenso de gradiente implementado a mano converge al óptimo exacto (diferencia máxima 0,012 en los coeficientes) y detecta su propia divergencia. Los modelos, sin embargo, explican poco (R^2 0,085) y la exactitud de la clasificación no supera la línea base, lo que confirma lo anticipado en la Fase 2: las variables institucionales no capturan las trayectorias individuales. Esa limitación es el punto de partida de la Fase 4.

13.4 Limitaciones y proyección

5. La cohorte de objetos y la matriz de modelación usan muestras aleatorias reproducibles (300.000 y 200.000 registros) por costo de instanciación; el diseño escala al total, pero el notebook prioriza tiempos de ejecución razonables.

6. Quickselect asigna tres listas por nivel; una variante in-place (partición de Hoare) reduciría el pico de memoria y acercaría el exponente empírico a 1.
7. Para F4: ampliar las variables del modelo, incorporar interacciones y modelos no lineales, y construir el reporte final sobre la jerarquía recursiva, que responde «dónde se concentra el rezago» a cualquier nivel con una sola llamada.

XIV. Bibliografía

Referencias en formato APA 7.ª edición, citadas en el cuerpo del informe y en el notebook.

Bibliografía docente

Contreras Bravo, L. E., & Padilla Beltrán, J. E. (2024). Ciencia de datos con Python: transformación y selección de variables. Ediciones de la U.

Grus, J. (2019). Data science from scratch: First principles with Python (2.ª ed.). O'Reilly Media.

Wickham, H., & Grolemund, G. (2017). R para la ciencia de datos: importar, ordenar, transformar, visualizar y modelar datos. O'Reilly Media. <https://es.r4ds.hadley.nz/>

Documentación técnica oficial

NumPy Developers. (2025). NumPy documentation (versión 2.5). <https://numpy.org/doc/stable/>

pandas Development Team. (2025). pandas documentation (versión 3.0). <https://pandas.pydata.org/docs/>

Python Software Foundation. (2025a). timeit — Measure execution time of small code snippets. <https://docs.python.org/3/library/timeit.html>

Python Software Foundation. (2025b). tracemalloc — Trace memory allocations. <https://docs.python.org/3/library/tracemalloc.html>

Python Software Foundation. (2025c). abc — Abstract base classes. <https://docs.python.org/3/library/abc.html>

Bibliografía complementaria

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.ª ed.). MIT Press.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E.

(2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.
<https://doi.org/10.1038/s41586-020-2649-2>

Ramalho, L. (2022). *Fluent Python: Clear, concise, and effective programming* (2.^a ed.). O'Reilly Media.

XV. Anexos

Anexo A. Instituciones con mayor sobreduración media en el árbol jerárquico

Nodos de nivel «institución» con al menos 1.000 titulados en la cohorte de 300.000, ordenados por sobreduración media. Fuente: reports/tables/f3_jerarquia_instituciones.csv.

Institución	Titulados	Sobreduración media	Titulación oportuna
Universidad Bolivariana	1.019	5,79 sem.	9,9 %
Universidad Tecnica Federico Santa Maria	3.809	5,43 sem.	2,7 %
Universidad Tecnologica de Chile Inacap	6.795	4,85 sem.	2,6 %
Universidad de la Serena	1.449	4,79 sem.	3,9 %
Cft Inacap	20.366	4,61 sem.	1,9 %
Universidad de Antofagasta	1.574	4,28 sem.	4,1 %
Universidad de Valparaíso	3.066	4,20 sem.	2,8 %
Universidad de Tarapaca	1.781	4,09 sem.	6,1 %

Anexo B. Salida completa de las pruebas automatizadas

===== test session starts =====
collecting ... collected 73 items
tests/test_nucleo.py::TestAgrupacion::test_medio_por_grupo coincide_con_pandas PASSED [1%]
tests/test_nucleo.py::TestAgrupacion::test_un_solo_grupo PASSED [2%]
tests/test_nucleo.py::TestAgrupacion::test_largos_distintos lanzan_error PASSED [4%]
tests/test_nucleo.py::TestOrdenamiento::test_merge_sort coincide_con_sorted PASSED [5%]
tests/test_nucleo.py::TestOrdenamiento::test_no_muta_la_entrada PASSED [6%]
tests/test_nucleo.py::TestOrdenamiento::test_vacia_y_un_elemento PASSED [8%]
tests/test_nucleo.py::TestOrdenamiento::test_es_estable PASSED [9%]
tests/test_nucleo.py::TestSeleccion::test_mediana_coincide_con_statistics PASSED [10%]
tests/test_nucleo.py::TestSeleccion::test_quickselect recupera_cada_posicion PASSED [12%]
tests/test_nucleo.py::TestSeleccion::test_mediana_par_e_impar PASSED [13%]
tests/test_nucleo.py::TestSeleccion::test_k_fuera_de_rango_y_vacia PASSED [15%]
tests/test_nucleo.py::TestBusqueda::test_binaria_encuentra_y_no_encuentra PASSED [16%]
tests/test_nucleo.py::TestBusqueda::test_binaria_en_vacia PASSED [17%]
tests/test_nucleo.py::TestBusqueda::test_conteos_por_umbral coinciden_con_recuento_directo PA...
tests/test_nucleo.py::TestBusqueda::test_cotas_en_bordes PASSED [20%]
tests/test_nucleo.py::TestBenchmark::test_medicion_basica PASSED [21%]
tests/test_nucleo.py::TestBenchmark::test_exponente_de_funcion_lineal cercano_a_uno PASSED [...
tests/test_nucleo.py::TestBenchmark::test_exponente_cuadratico PASSED [24%]
tests/test_nucleo.py::TestBenchmark::test_estimar_orden exige_dos_tamano PASSED [26%]
tests/test_nucleo.py::TestBenchmark::test_comparar_ordena_por_tiempo PASSED [27%]
tests/test_nucleo.py::TestTitulado::test_propiedades_derivadas PASSED [28%]
tests/test_nucleo.py::TestTitulado::test_encapsulamiento_identidad_solo_lectura PASSED [30%]
tests/test_nucleo.py::TestTitulado::test_setters_validan PASSED [31%]
tests/test_nucleo.py::TestTitulado::test_orden_natural_e_igualdad PASSED [32%]
tests/test_nucleo.py::TestTitulado::test_desde_fila_con_dict_y_series PASSED [34%]

tests/test_nucleo.py::TestCohorte::test_protocolo_de_secuencia	PASSED	[35%]
tests/test_nucleo.py::TestCohorte::test_filtrar_y_agrupar_no_mutan	PASSED	[36%]
tests/test_nucleo.py::TestCohorte::test_strategy_con_metricas	PASSED	[38%]
tests/test_nucleo.py::TestCohorte::test_ordenados_con_merge_sort	PASSED	[39%]
tests/test_nucleo.py::TestCohorte::test_agregar_rechaza_tipos_ajenos	PASSED	[41%]
tests/test_nucleo.py::TestCohorte::test_metrica_en_cohorte_vacia_lanza_error	PASSED	[42%]
tests/test_nucleo.py::TestCohorte::test_desde_dataframe_exige_columnas	PASSED	[43%]
tests/test_nucleo.py::TestCohorte::test_percentil_valida_rango	PASSED	[45%]
tests/test_nucleo.py::TestJerarquia::test_totales_rekursivos_y_estructura	PASSED	[46%]
tests/test_nucleo.py::TestJerarquia::test_cache_se_invalida_al_insertar	PASSED	[47%]
tests/test_nucleo.py::TestJerarquia::test_metrica_composite_y_descenso	PASSED	[49%]
tests/test_nucleo.py::TestJerarquia::test_buscar_y_dataframe	PASSED	[50%]
tests/test_nucleo.py::TestJerarquia::test_nodo_vacio	PASSED	[52%]
tests/test_nucleo.py::TestPreparacion::test_escalador_estandar	PASSED	[53%]
tests/test_nucleo.py::TestPreparacion::test_estandar_invertir	PASSED	[54%]
tests/test_nucleo.py::TestPreparacion::test_minmax_en_cero_uno	PASSED	[56%]
tests/test_nucleo.py::TestPreparacion::test_transformar_sin_ajustar_y_dimensiones	PASSED	[57%]
tests/test_nucleo.py::TestPreparacion::test_onehot_omite_primera_y_categoria_desconocida	PASS...	
tests/test_nucleo.py::TestPreparacion::test_particion_reproducibile_y_disjunta	PASSED	[60%]
tests/test_nucleo.py::TestPreparacion::test_preparador_compone_y_exige_ajuste	PASSED	[61%]
tests/test_nucleo.py::TestModelos::test_descenso_converge_a_solucion_cerrada	PASSED	[63%]
tests/test_nucleo.py::TestModelos::test_perdida_decrece	PASSED	[64%]
tests/test_nucleo.py::TestModelos::test_logistica_separa_datos_separables	PASSED	[65%]
tests/test_nucleo.py::TestModelos::test_pesos_devueltos_son_copia	PASSED	[67%]
tests/test_nucleo.py::TestModelos::test_errores_controlados	PASSED	[68%]
tests/test_nucleo.py::TestModelos::test_metricas_clasificacion_extremos	PASSED	[69%]
tests/test_pipeline.py::test_duracion_real_caso_normal	PASSED	[71%]
tests/test_pipeline.py::test_sobreduracion_y_categoria	PASSED	[72%]
tests/test_pipeline.py::test_edad_descuenta_mes_no_cumplido	PASSED	[73%]
tests/test_pipeline.py::test_edad_cuando_aun_no_cumple_anios	PASSED	[75%]
tests/test_pipeline.py::test_semestre_de_titulacion_según_mes	PASSED	[76%]
tests/test_pipeline.py::test_duracion_minima_es_uno	PASSED	[78%]
tests/test_pipeline.py::test_clasificacion_en_los_bordes	PASSED	[79%]
tests/test_pipeline.py::test_sentinelas_se_convierten_en_na	PASSED	[80%]
tests/test_pipeline.py::test_normalizacion_de_texto_respetar_nombres_propios	PASSED	[82%]
tests/test_pipeline.py::test_duplicados_sin_mrun_no_se_colapsan	PASSED	[83%]
tests/test_pipeline.py::test_duplicado_exacto_se_elimina	PASSED	[84%]
tests/test_pipeline.py::test_duracion_teorica_cero_no_divide_por_cero	PASSED	[86%]
tests/test_pipeline.py::test_registro_sin_fecha_nacimiento_se_marca_atipico	PASSED	[87%]
tests/test_pipeline.py::test_archivos_faltantes_lanza_excepcion	PASSED	[89%]
tests/test_pipeline.py::test_cargar_consolidado_inexistente	PASSED	[90%]
tests/test_pipeline.py::test_validador_detecta_dataset_corrupto	PASSED	[91%]
tests/test_pipeline.py::test_validador_aprueba_dataset_correcto	PASSED	[93%]
tests/test_pipeline.py::test_estado_antes_de_ejecutar_lanza_error	PASSED	[94%]
tests/test_pipeline.py::test_regla_con_excepcion_se_reporta_como_falla	PASSED	[95%]
tests/test_pipeline.py::test_tabla_agregada_respetar_minimo_de_casos	PASSED	[97%]
tests/test_pipeline.py::test_formato_numerico_chileno	PASSED	[98%]
tests/test_pipeline.py::test_configuracion_expone_versiones	PASSED	[100%]
===== 73 passed in 3.18s =====		

Anexo C. Contribuciones y ramas

git shortlog -sn --all

32 Sebastian Antunez

3 santunez75

Anexo D. Reproducción de la fase

Comandos para reproducir la Fase 3 en cualquier equipo

```
git clone https://github.com/santunez75/proyecto-titulados.git
cd proyecto-titulados
git checkout fase-3/nucleo-algoritmico
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m pytest tests/ -q          # 73 pruebas, sin datos externos
.venv\Scripts\python.exe -m src.pipeline              # opcional: regenera el dataset de F2
.venv\Scripts\python.exe -m jupyter lab F3/F3_Nucleo_Algoritmico.ipynb
```